DP Bottom Up

Técnicas de Diseño de Algoritmos

FCEyN UBA

October 8, 2026

Lo que veremos hoy

Introcucción y Metodología
- Problema de la Mochila

Motivación
- Fibonacci

Ejercicios teórica
- Varillas
- Producto de Matrices
- Conjunto Independiente Máximo

Ejercicios de la Guía
- OptiPago
- César

Lo que veremos hoy

Introcucción y Metodología
- Problema de la Mochila

Motivación
- Fibonacci

Ejercicios teórica
- Varillas
- Producto de Matrices
- Conjunto Independiente Máximo

Ejercicios de la Guía
- OptiPago
- César

Problema de la mochila

Definición

Tenemos n objetos. El objeto i tiene peso entero $a_i > 0$ y valor $b_i \geq 0$. La capacidad W es un entero no negativo. Buscamos

$$\max_{S \subseteq \{1,\ldots,n\}} \sum_{i \in S} b_i \qquad \text{sujeto a } \sum_{i \in S} a_i \leq W.$$

Objeto i	1	2	3
Peso a_i	3	2	2
Valor b_i	5	3	4

Capacidad $W = 5$.
Elegir $S = \{1, 3\}$ ocupa peso 5 y obtiene valor 9.

El problema de la mochila: con backtracking

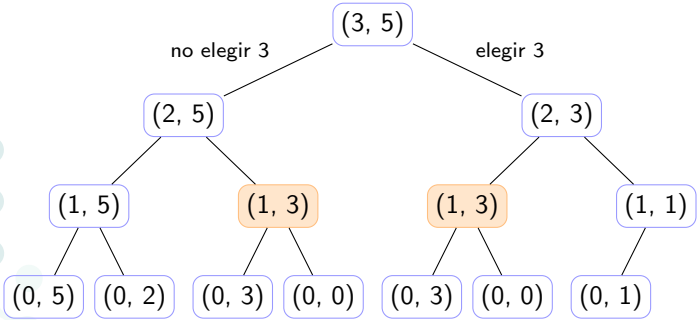
Ya sabemos resolver este problema con una función recursiva.

$$F(i,w) = \begin{cases} -\infty, & w < 0, \\ 0, & i = 0, \\ \max\{F(i-1,w), b_i + F(i-1, w-a_i)\}, & i > 0. \end{cases}$$

La respuesta a la instancia original es $F(n, W)$. El algoritmo tiene complejidad de peor caso $O(2^n)$.

El problema de la mochila: el árbol de backtracking

Podemos asociar a cada nodo del árbol los parámetros de su llamado recursivo. Luego, hay casos en los que varios nodos están asociados a llamados idénticos.



Ambos nodos hacen **exactamente** los mismos cálculos. La idea entonces es guardar el resultado cuando se procesa uno de estos nodos para ya tener el valor listo cuando se procesa el segundo.

El problema de la mochila: memorizar las soluciones

Cada nodo se asocia a un conjunto de parámetros de la función, y la función toma dos naturales. Luego, podemos usar una matriz para guardar los valores.

Estructura de memorización

Usamos una matriz $M[0..n, 0..W]$. La idea es que $M[i, w]$ guarde $F(i, w)$ una vez que se lo calcule.

- Con esta estructura consultar o escribir una celda cuesta $O(1)$.
- Inicialmente, todas las celdas están en un valor nulo $\perp$.

El problema de la mochila: backtracking normal

$\mathrm{MOCHILA}(i, w)$
  si $w < 0$:
    devolver $-\infty$
  si $i = 0$:
    devolver 0
  devolver $\max(\mathrm{MOCHILA}(i-1, w), \mathrm{MOCHILA}(i-1, w-a_i) + b_i)$

El problema de la mochila: backtracking con memoria

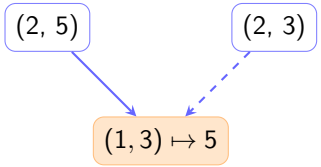
MOCHILA-PD(i, w)
  si $w < 0$:
    devolver $-\infty$
  si $i = 0$:
    devolver 0
  si $M[i, w] = \perp$:
    $M[i, w] \leftarrow \max(\text{MOCHILA-PD}(i-1, w), \text{MOCHILA-PD}(i-1, w-a_i) + b_i)$
  devolver $M[i, w]$

Antes de ejecutar MOCHILA-PD(n, W) hay que inicializar la memoria con $\perp$ en todas las posiciones.

Ejemplo de memorización

En el árbol de ejemplo que vimos antes, la memoria se va a usar para no recalcular el nodo con valor $(1, 3)$

1. Desde $(2, 5)$ se calcula $F(1, 3) = 5$.
2. Antes de volver, se escribe $M[1, 3] = 5$.
3. Desde $(2, 3)$ se vuelve a pedir $F(1, 3)$.
4. La consulta devuelve 5 sin hacer recursión.



Mismo subproblema!

Cuándo ayuda la memoria?

En el fondo, ahora la complejidad va a depender de la *cantidad de estados distintos*, ya que estados con misma etiqueta se procesan una única vez.

Pensemos mejor esto último.

Cada estado es visitado a lo sumo dos veces

Un estado (i, w), con $i < n$, solo puede ser llamado desde:

$$\underbrace{(i+1, w)}_{\text{no elegir } i+1} \quad \text{o} \quad \underbrace{(i+1, w + a_{i+1})}_{\text{elegir } i+1}$$

Teniendo en cuenta que cada nodo se expande una única vez, concluímos que cada nodo es procesado a lo sumo dos veces.

El costo total incluye visitas e inicialización

- Ya sabemos que cada nodo se visita una única vez. Ahora... Cuántos nodos distintos hay? Cómo hay nW etiquetas distintas, a lo sumo se van a visitar nW nodos distintos.
- Cada visita hace $O(1)$ operaciones. Luego, procesar todos los nodos toma $O(nW)$.
- La matriz de memoria tiene $O(nW)$ posiciones, y llenarla al principio toma $O(nW)$.

La complejidad final entonces es... $O(nW)$

Reconstrucción de la solución

Para la instancia que vimos del problema de la mochila teníamos que $F(3, 5) = 9$.
Pero... Y si queremos encontrar una solución?

> Vamos a ver que podemos adaptar el algoritmo que calcula el valor óptimo para que también devuelva la solución.

Esto sugiere el siguiente principio de diseño: nos olvidamos de la construcción, y solo pensamos en cómo calcular el valor óptimo. Luego, aplicamos estas técnicas de reconstrucción para recuperar la solución.

Reconstrucción

Si en (i, w), con $i > 0$, y conocemos $M[i, w] = F(i, w)$... Como redescubrimos la decisión que hizo el backtracking?

Si $M[i, w] = M[i - 1, w]$

Podemos omitir el objeto i y pasar a $(i - 1, w)$ sin perder valor.

Si $M[i, w] > M[i - 1, w]$

La alternativa óptima incluye al objeto i. Entonces $M[i, w] = b_i + M[i - 1, w - a_i]$. Agregamos i a la solución y seguimos reconstruyendo a partir del estado $(i - 1, w - a_i)$.

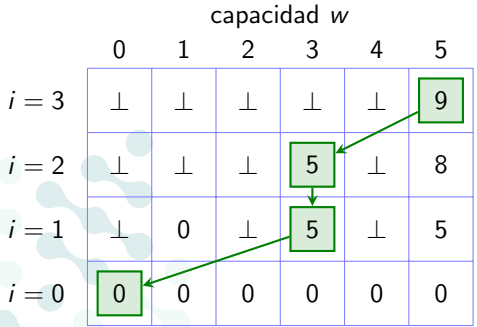
En cada paso disminuye i: por lo que en n pasos terminamos. En qué complejidad se implementa esta estrategia? $O(n)$.

Ejemplo de reconstrucción

RECONSTRUIR-MOCHILA(M, n, W)
  $L \leftarrow$ lista vacía; $i \leftarrow n$; $w \leftarrow W$
  while $i > 0$
    si $M[i, w] > M[i-1, w]$:
      agregar i al final de L
      $w \leftarrow w - a_i$
    $i \leftarrow i - 1$
  end while
  devolver L

- Ejecutar después de MOCHILA-PD(n, W).
- Hace falta tener la memoria!

Ejemplo de reconstrucción



Objeto	1	2	3
Peso	3	2	2
Valor	5	3	4

$W = 5$; óptimo 9.
En verde: estados del camino.
$\perp$: celda no calculada.

La figura muestra la tabla que deja MOCHILA-PD$(3, 5)$, con la fila del fondo inicializada con el caso base.

Costos innecesarios

- **Inconveniente con la recursividad:** cada llamada recursiva copia a la pila los parámetros y las variables del contexto y cada vuelta los repone.
- Eso tiene un costo. Es solo constante pero tenemos una forma mecánica de eliminarlo.
- Podemos también ahorrar espacio en muchos casos (lo vemos luego).

Consigna

¿Cómo podemos modificar el algoritmo de la mochila para que sea iterativo?

Idea: llenar toda la matriz cache.

Llenando la cache

- ¿Qué se guarda en cada celda de la cache?
  $M[i, w] = \text{MOCHILA-PD}(i, w) \quad (\forall i, w)$
- Casos base: $M[0, w] = 0 \quad \forall w$
- Casos recursivos: $(\forall i > 0, w \geq a_i)$
  $M[i, w] = \text{MOCHILA-PD}(i, w) =$
  $\max(\text{MOCHILA-PD}(i-1, w), \text{MOCHILA-PD}(i-1, w-a_i) + b_i) =$
  $\max(M[i-1, w], M[i-1, w-a_i] + b_i)$
- Alcanza con llenar todas las celdas de acuerdo a esta igualdad.
- ¿En qué orden?
- Cualquiera que garantice que $M[i-1, w]$ y $M[i-1, w-a_i]$ estén calculadas al momento de computar $M[i, w]$

Algoritmo bottom-up

MOCHILA-BU(n, W)
  para $j \leftarrow 0, \ldots, W$
    $M[0, j] \leftarrow 0$
  para $i \leftarrow 1, \ldots, n$
    para $w \leftarrow 0, \ldots, W$
      $M[i, w] \leftarrow M[i-1, w]$
      si $w \geq a_i$:
        $M[i, w] \leftarrow \max(M[i, w], M[i-1, w-a_i] + b_i)$
  devolver $M[n, W]$

¿Complejidad?
Claramente dada por la cantidad de celdas y el costo de llenar cada una

Optimización de espacio

¿Necesitamos toda la matriz?

- Solo la fila anterior
  Se puede guardar solo dos arreglos: `fila_actual` y `fila_anterior`
- Al completar `fila_actual` esta reemplaza a `fila_anterior` y se comienza a computar una nueva `fila_actual`.

Advertencia

Esto solo es posible si no reconstruiremos la solución. Si no, es necesario recorrer un camino a través de toda la matriz.

Algoritmo bottom-up con optimización de espacio

MOCHILA-BU-OPT(n, W)

$F_{ant} \leftarrow$ arreglo de tamaño $W+1$ inicializado en 0
$F_{act} \leftarrow$ arreglo de tamaño $W+1$ inicializado en 0
para $i \leftarrow 1, \ldots, n$
  para $w \leftarrow 0, \ldots, W$
    $F_{act}[w] \leftarrow F_{ant}[w]$ ▷ Decisión: no elegir el objeto
    si $w \geq a_i$:
      $F_{act}[w] \leftarrow \max(F_{act}[w], F_{ant}[w - a_i] + b_i)$
  $F_{ant} \leftarrow$ copia de F_{act} ▷ Avanzamos a la siguiente fila
devolver $F_{ant}[W]$

Complejidad Final:
Tiempo: $\Theta(nW)$ (mantenemos el doble ciclo iterativo).
Espacio: $\Theta(W)$ (reducimos la memoria cuadrática a solo dos arreglos lineales).

Lo que veremos hoy

Introcucción y Metodología
- Problema de la Mochila

Motivación
- Fibonacci

Ejercicios teórica
- Varillas
- Producto de Matrices
- Conjunto Independiente Máximo

Ejercicios de la Guía
- OptiPago
- César

Motivación: Fibonacci

$$F(n) = \begin{cases} n & \text{si } n \leq 1, \\ F(n-1) + F(n-2) & \text{si } n \geq 2. \end{cases}$$

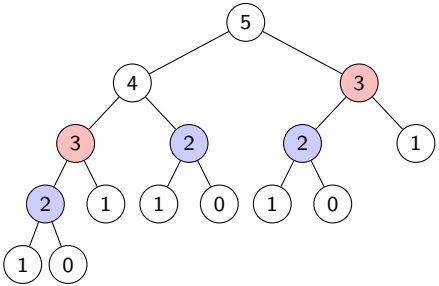
n	0	1	2	3	4	5	6	7	8
$F(n)$	0	1	1	2	3	5	8	13	21

Vamos a resolverlo de tres formas:

1. Recursivo sin memoria (backtracking).
2. Recursivo con memoria (*top-down*).
3. Iterativo llenando una tabla (*bottom-up*).

Fibonacci recursivo

```
FIB(n)
  si n ≤ 1 :
      devolver n
  a ← FIB(n − 1)
  b ← FIB(n − 2)
  devolver a + b
```



- $F(3)$ se calcula 2 veces, $F(2)$ 3 veces. Para n grande, muchísimas más.
- Complejidad exponencial

Fibonacci top-down

Vector $M[0..n]$ inicializado en $\perp$.

```
FIB-MEM(n)
  si n ≤ 1 :
      devolver n
  si M[n] ≠ ⊥ :
      devolver M[n]
  M[n] ← FIB-MEM(n − 1)
         + FIB-MEM(n − 2)
  devolver M[n]
```

- Misma recursión, pero guardamos cada resultado la primera vez que lo calculamos.
- Hay $n + 1$ estados distintos y cada uno se procesa una única vez, en $O(1)$.

Tiempo: $\Theta(n)$
Espacio adicional: $\Theta(n)$

De exponencial a lineal, sólo agregando memoria.

Fibonacci bottom-up

```
FIB-BU(n)
  M[0] ← 0
  M[1] ← 1
  para i = 2, ..., n
      M[i] ← M[i − 1] + M[i − 2]
  devolver M[n]
```

- $F(i)$ sólo depende de valores con índice menor.
- Llenando M de menor a mayor, todo lo que necesitamos ya está calculado.
- Sin recursión: un ciclo que lee de la tabla.

Tiempo: $\Theta(n)$
Espacio adicional: $\Theta(n)$

Para pensar

¿Hace falta guardar todo el vector? Sólo usamos los dos últimos valores: se puede hacer con espacio $O(1)$.

Fibonacci bottom-up con espacio $O(1)$

```
FIB-BU-O1(n)
  si n ≤ 1 :
      devolver n
  a ← 0                    ▷ F(i − 2)
  b ← 1                    ▷ F(i − 1)
  para i = 2, . . . , n
      c ← a + b
      a ← b
      b ← c
  devolver b
```

- Para calcular $F(i)$ sólo necesitamos $F(i-1)$ y $F(i-2)$.
- En vez de todo el vector, guardamos los dos últimos valores y los vamos desplazando.

Tiempo: $\Theta(n)$
Espacio adicional: $O(1)$
A veces Bottom Up nos permite ahorrar memoria comparado con Top Down.

Lo que veremos hoy

Introcucción y Metodología
- Problema de la Mochila

Motivación
- Fibonacci

Ejercicios teórica
- Varillas
- Producto de Matrices
- Conjunto Independiente Máximo

Ejercicios de la Guía
- OptiPago
- César

Enunciado Varillas

Tenemos una varilla de longitud entera $n \geq 1$ y precios $p_1, \ldots, p_n$: una pieza de longitud k se vende por p_k.

Objetivo

Queremos dividir la varilla en piezas de longitudes enteras positivas que sumen n, de modo de maximizar el ingreso total. Se permite venderla sin cortar, y los cortes en sí no tienen costo.

longitud k	1	2	3	4
precio p_k	1	5	8	9

Una varilla de longitud 4 se podría vender por 9 pesos o bien se la podría partir en dos partes y vender a 5 cada una.

Enunciado varillas

1. Repasemos cómo sería la funcion recursiva y la solución Top Down (visto en la teórica)
2. Resolverlo Bottom Up
3. Dar la complejidad temporal y de memoria

Función recursiva

Llamemos $R(\ell)$ al mayor ingreso que podemos obtener de una varilla de longitud ℓ.

$$R(\ell) = \begin{cases} 0 & \text{si } \ell = 0, \\ \max\limits_{1 \le k \le \ell} \{ p_k + R(\ell - k) \} & \text{si } \ell > 0. \end{cases}$$

- Vendemos una primera pieza de longitud k por p_k.
- Resolvemos recursivamente el resto, de longitud $\ell - k$.
- La llamada inicial es $R(n)$.

Si implementamos con Backtracking, la complejidad es exponencial (ver diapos teórica).

Solución top-down

Usamos un vector $M[0..n]$ inicializado en $\perp$.

```
CORTAR-MEM(ℓ)
  si ℓ = 0 :
      devolver 0
  si M[ℓ] ≠ ⊥ :
      devolver M[ℓ]
  q ← −∞
  para k = 1, . . . , ℓ
      r ← CORTAR-MEM(ℓ − k)
      q ← max{q, p_k + r}
  M[ℓ] ← q
  devolver q
```

Complejidad

- Inicializar M: $O(n)$.
- Hay n estados distintos y cada uno se procesa una única vez.
- El estado ℓ hace ℓ iteraciones, cada una $O(1)$ si el valor ya está en memoria.

Tiempo: $\Theta(n^2)$
Espacio adicional: $\Theta(n)$

Idea bottom-up

- $R(\ell)$ sólo depende de $R(0), R(1), \ldots, R(\ell-1)$.
- Si completamos M en orden creciente de ℓ, cuando llegamos a $M[\ell]$ todo lo que necesitamos ya está calculado.
- Reemplazamos la recursión por un ciclo que lee de la tabla:

$$M[\ell] = \max_{1 \le k \le \ell} \{\, p_k + M[\ell - k] \,\}$$

ℓ	opciones $p_k + M[\ell-k]$	$M[\ell]$
0	caso base	0
1	$1+0$	1
2	$1+1,\ 5+0$	5
3	$1+5,\ 5+1,\ 8+0$	8
4	$1+8,\ 5+5,\ 8+1,\ 9+0$	**10**

Solución bottom-up

```
CORTAR-BU(n)
  M[0] ← 0
  para ℓ = 1, ..., n
      q ← −∞
      para k = 1, ..., ℓ
          q ← max{q, p_k + M[ℓ − k]}
      M[ℓ] ← q
  devolver M[n]
```

Complejidad

- El ciclo externo da n vueltas.
- En la vuelta ℓ, el ciclo interno da ℓ vueltas de costo $O(1)$.

$$\sum_{\ell=1}^{n} \ell = \frac{n(n+1)}{2} = \Theta(n^2)$$

Tiempo: $\Theta(n^2)$
Espacio adicional: $\Theta(n)$

Igual que top-down!

Lo que veremos hoy

Introcucción y Metodología
- Problema de la Mochila

Motivación
- Fibonacci

Ejercicios teórica
- Varillas
- Producto de Matrices
- Conjunto Independiente Máximo

Ejercicios de la Guía
- OptiPago
- César

Producto de Matrices: Enunciado y Recursión

Queremos calcular el producto de una cadena de matrices $A_1 A_2 \dots A_n$, donde A_i tiene tamaño $d_{i-1} \times d_i$. Buscamos parentizar para minimizar las multiplicaciones escalares.

Definimos $m(i, j)$ como el costo mínimo de multiplicar $A_i \dots A_j$:

$$m(i,j) = \begin{cases} 0 & \text{si } i = j \\ \min\limits_{i \le k < j} \{m(i,k) + m(k+1,j) + d_{i-1} d_k d_j\} & \text{si } i < j \end{cases}$$

La respuesta a nuestro problema se encuentra calculando $m(1, n)$.

Producto de Matrices: Backtracking vs. Top-Down

Recursión pura (Backtracking)

- Recalcula los mismos subproblemas múltiples veces (árbol de recursión enorme).
- **Tiempo:** $\Theta(n3^n)$ (Exponencial).
- **Espacio:** $\mathcal{O}(n)$.

Con Memoria (Top-Down)

- Memoizamos: cada estado (i, j) se procesa una única vez.
- **Tiempo:** $\Theta(n^3)$ (Polinomial).
- **Espacio:** $\Theta(n^2)$.

Conclusión

¡Agregar una matriz de memoria mejoró exponencialmente el algoritmo!

¡A pensar!

Consigna:

1. Intenten escribir el algoritmo iterativo (Bottom-Up) para llenar la matriz M.
2. *Pista:* Observen la fórmula recursiva. Para calcular el intervalo $[i, j]$, ¿qué intervalos necesitamos tener calculados previamente? ¿En qué orden debemos llenar la tabla?
3. Calculen la complejidad temporal y espacial de su algoritmo iterativo.

Producto de Matrices: Top-Down vs. Bottom-Up

Enfoque Top-Down

- Usamos una matriz $M[1..n, 1..n]$ inicializada en $\perp$.
- **Estados distintos:** $\Theta(n^2)$
- **Costo por llamado:** $\mathcal{O}(n)$
- **Tiempo:** $\Theta(n^3)$
- **Espacio adicional:** $\Theta(n^2)$

Enfoque Bottom-Up (Idea)

- Queremos llenar la tabla M de forma iterativa.
- Para calcular $M[i, j]$, necesitamos que $M[i, k]$ y $M[k + 1, j]$ ya estén calculados.
- ¿En qué orden debemos iterar para garantizar esto?

¡Por la longitud del intervalo (la subcadena)!

Entendiendo las Dependencias

- **¿Qué guarda cada casillero?** La celda $M[i, j]$ guarda el costo mínimo de la subcadena $A_i \dots A_j$. Su longitud es $L = j - i + 1$.
- **¿De qué depende?** Para calcularla, probamos cortarla en un punto k. Usamos el costo de la **mitad izquierda** ($M[i, k]$) y la **mitad derecha** ($M[k + 1, j]$).
- **El concepto clave:** Al partir una cadena en dos, ¡las dos partes resultantes son siempre estrictamente más cortas que la cadena original!
- **¿Por qué sirve este enfoque?** Si calculamos primero todas las cadenas de longitud 1, luego las de longitud 2, y así sucesivamente, **te garantizás** que cuando necesites armar una cadena de longitud L, todas sus posibles mitades (que son más cortas) ya van a estar calculadas en la tabla.

El Orden de Llenado de la Tabla

Llenaremos la matriz M iterando por la **longitud** L de la cadena:

1. $L = 1$: Cadenas de 1 matriz (diagonal principal, costo 0).
2. $L = 2$: Pares contiguos (diagonal superior).
3. $L = 3$: Tríos contiguos.
4. ... hasta llegar a $L = n$, que es $M[1, n]$.

$L = 1$	$L = 2$	$L = 3$		$L = n$
	$L = 1$	$L = 2$	$L = 3$	
		$L = 1$	$L = 2$	$L = 3$
			$L = 1$	$L = 2$
				$L = 1$

Llenado por diagonales

Solución Bottom-Up

```
MATRICES-BU(d, n)
  para i = 1, ..., n
    M[i, i] ← 0                                   ▷ Casos base: longitud 1
  para L = 2, ..., n                             ▷ Iteramos por longitud de subcadena
    para i = 1, ..., n − L + 1                    ▷ Posición de inicio
      j ← i + L − 1                               ▷ Posición de fin
      M[i, j] ← +∞
      para k = i, ..., j − 1                      ▷ Probamos todos los cortes
        costo ← M[i, k] + M[k + 1, j] + d_{i−1} · d_k · d_j
        M[i, j] ← min(M[i, j], costo)
  devolver M[1, n]
```

Entendiendo el código: Explorando los cortes

Supongamos que queremos calcular $M[1,4]$ (cadena de longitud $L=4$).
El ciclo interno prueba todos los cortes k posibles entre $i=1$ y $j=4$:

- **Corte $k=1$:** $\underbrace{(A_1)}_{M[1,1]} \cdot \underbrace{(A_2A_3A_4)}_{M[2,4]} \implies costo = M[1,1] + M[2,4] + d_0d_1d_4$
- **Corte $k=2$:** $\underbrace{(A_1A_2)}_{M[1,2]} \cdot \underbrace{(A_3A_4)}_{M[3,4]} \implies costo = M[1,2] + M[3,4] + d_0d_2d_4$
- **Corte $k=3$:** $\underbrace{(A_1A_2A_3)}_{M[1,3]} \cdot \underbrace{(A_4)}_{M[4,4]} \implies costo = M[1,3] + M[4,4] + d_0d_3d_4$

La clave del Bottom-Up

¡Observen los subíndices de las matrices! Para resolver un problema de longitud 4, necesitamos consultar celdas de longitudes 1, 2 y 3. Como llenamos la tabla iterando explícitamente por la longitud L, todas esas celdas ya fueron calculadas y están listas para usarse.

Entendiendo el código: Eligiendo el mejor corte

Como la celda $M[1,4]$ arranca valiendo $+\infty$, la instrucción iterativa:

`M[i,j]` $\leftarrow \min($`M[i,j], costo`$)$

compara las tres alternativas ($k = 1, 2, 3$) a medida que el ciclo avanza y se queda **únicamente con el valor del corte más barato**.

En la matriz:
Para cada corte, $M[1,4]$ suma dos valores que ya están fijos en la tabla.

$M[1,1]$	$M[1,2]$	$M[1,3]$	$M[1,4]$
	$M[2,2]$	$M[2,3]$	$M[2,4]$
		$M[3,3]$	$M[3,4]$
			$M[4,4]$

Dependencias para probar $M[1,4]$

Complejidad: Análisis Final

Cálculo de Complejidad Bottom-Up

Al igual que en Top-Down, analizamos los estados:

- Tenemos $\Theta(n^2)$ estados distintos (celdas de la matriz a llenar).
- Para calcular cada estado, el ciclo interno (cortes k) realiza a lo sumo n iteraciones de costo $\mathcal{O}(1)$. Es decir, el **trabajo por estado** es $\mathcal{O}(n)$.

Tiempo total: $\Theta(n^2)$ estados $\times$ $\mathcal{O}(n)$ trabajo por estado $= \Theta(n^3)$
Espacio adicional: $\Theta(n^2)$ (para la matriz M)

Una nota final sobre el uso de la memoria

La limitación de memoria en Matrices

A diferencia de Fibonacci (donde bajamos la memoria a $\mathcal{O}(1)$), para calcular cada diagonal L de matrices necesitamos consultar subcadenas de **todos** los tamaños menores, por ende requerimos la matriz entera en memoria.

¿Llenamos toda la matriz?

¡No! Como los intervalos solo tienen sentido si $i \leq j$, únicamente completamos la **mitad superior** de la matriz (un triángulo de aproximadamente $\frac{n^2}{2}$ celdas).

Aunque en la práctica esto guarda la mitad de los datos, en notación asintótica las constantes se ignoran, por lo que el espacio sigue siendo rigurosamente $\Theta(n^2)$.

Bonus: Otras formas válidas de llenar la tabla

- **La regla de oro:** Para calcular $M[i, j]$, necesitamos datos de su **izquierda** ($M[i, k]$) y de **abajo** ($M[k + 1, j]$).
- Cualquier recorrido que garantice que esas celdas ya estén calculadas, ¡es válido!

1. Por Filas (de abajo hacia arriba)

Avanzamos desde la última fila hacia la primera, recorriendo cada fila de izquierda a derecha.

```
para i = n − 1 bajando hasta 1
    para j = i + 1, . . . , n
        M[i, j] ← +∞
        para k = i, . . . , j − 1
            ▷ calculo costo y min
```

2. Por Columnas (de izq a der)

Avanzamos desde la 2da columna hacia la n, recorriendo cada columna de abajo hacia arriba.

```
para j = 2, . . . , n
    para i = j − 1 bajando hasta 1
        M[i, j] ← +∞
        para k = i, . . . , j − 1
            ▷ calculo costo y min
```

Lo que veremos hoy

Introcucción y Metodología
- Problema de la Mochila

Motivación
- Fibonacci

Ejercicios teórica
- Varillas
- Producto de Matrices
- Conjunto Independiente Máximo

Ejercicios de la Guía
- OptiPago
- César

Conjunto independiente máximo

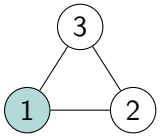
Dado un grafo simple no dirigido $G = (V, E)$, queremos un conjunto $I \subseteq V$ de máxima cardinalidad tal que

$$\text{para todo } u, v \in I : \ \{u, v\} \notin E.$$

Al tamaño de este conjunto se lo denota $\alpha(G)$.



$I = \{1, 3, 5\}$ es óptimo: $\alpha(G) = 3$.



Triángulo: $\alpha(G) = 1$.

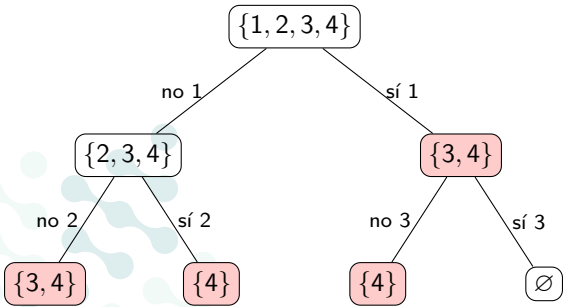
Función recursiva

Para $U \subseteq V$ (los vértices que todavía podemos usar), $F(U)$ es el tamaño del mayor conjunto independiente usando sólo vértices de U. La respuesta es $F(V)$.
Notamos $N_G[v] = \{v\} \cup \{\text{vecinos de } v\}$. Tomamos un vértice $v \in U$ y hay sólo dos opciones:

$$F(U) = \begin{cases} 0 & \text{si } U = \varnothing, \\ \max\{\ \underbrace{F(U \setminus \{v\})}_{\text{no elijo } v},\ \underbrace{1 + F(U \setminus N_G[v])}_{\text{elijo } v}\ \} & \text{si } U \neq \varnothing. \end{cases}$$

- **No elijo v:** lo descarto y sigo con el resto.
- **Elijo v:** sumo 1, y sus vecinos ya no pueden entrar: los saco junto con v.
- Sirve cualquier $v \in U$: todo conjunto independiente contiene a v o no lo contiene. Tomamos $v = \min U$.

Ejemplo: el camino 1–2–3–4



- Elegir 1 saca a 1 y a su vecino 2.
- $F(\{3,4\}) = 1$, $F(\{2,3,4\}) = 2$.
- $F(\{1,2,3,4\}) = \max\{2,\ 1+1\} = 2$.
- $F(\{3,4\})$ y $F(\{4\})$ se calculan más de una vez: memorizar parece buena idea.

Solución top-down

M: diccionario indexado por conjuntos, inicialmente vacío.

```
INDEP-MEM(U)
  si U = ∅ :
      devolver 0
  si U ∈ M :                                  ▷ O(n)
      devolver M[U]
  v ← min U                                   ▷ O(n)
  a ← INDEP-MEM(U \ {v})                      ▷ O(n)
  b ← 1+ INDEP-MEM(U \ N_G[v])                ▷ O(n)
  M[U] ← max{a, b}                            ▷ O(n)
  devolver M[U]
```

¿Por qué cada paso es $O(n)$?

- Buscar o guardar U en M: la clave es un conjunto de hasta n vértices, y hay que hashearla o compararla.
- $\min U$: recorrer U (con otra estructura puede ser $O(1)$, pero no cambia el total).
- Armar $U \setminus \{v\}$ y $U \setminus N_G[v]$: cada llamado necesita su propia copia de U, y copiar cuesta $O(n)$.

¿Sirvió memorizar?

- Cada estado se procesa una única vez, en $O(n)$.
- Pero los estados son subconjuntos de V: puede haber hasta 2^n. Y cada subconjunto puede tener n elementos.

Algoritmo	Tiempo	Memoria adicional
Recursión sin memoria	$O(n\,2^n)$	$O(n)$
Top-down	$O(n\,2^n)$	$O(n\,2^n)$

- Mismo tiempo, y encima memoria exponencial.
- En varillas el estado era un número $\ell \in \{0, \ldots, n\}$: pocos estados que se repetían muchísimo.
- Acá el estado tiene que arrastrar todo el conjunto U: hay poca superposición de subproblemas.

Idea bottom-up

- ¿En qué orden calculamos los estados? $F(U)$ depende de $F(U \setminus \{v\})$ y $F(U \setminus N_G[v])$.
- Ambos son subconjuntos estrictos de U: tienen menos elementos.
- Si calculamos los conjuntos de menor a mayor tamaño, cuando llegamos a U todo lo que necesitamos ya está calculado.

Tamaño	Conjuntos (para $V = \{1, 2, 3\}$)
0	$\varnothing$
1	$\{1\},\ \{2\},\ \{3\}$
2	$\{1, 2\},\ \{1, 3\},\ \{2, 3\}$
3	$\{1, 2, 3\}$

Solución bottom-up

M: diccionario indexado por conjuntos.

```
INDEP-BU(G)
  M[∅] ← 0
  para s = 1, ..., n
    para cada U ⊆ V con |U| = s
      v ← min U                          ▷ O(n)
      a ← M[U \ {v}]                     ▷ O(n)
      b ← 1 + M[U \ N_G[v]]              ▷ O(n)
      M[U] ← max{a, b}                   ▷ O(n)
  devolver M[V]
```

Complejidad

- Mismas operaciones que en top-down: cada una manipula un conjunto de hasta n vértices, $O(n)$.
- 2^n estados, cada uno $O(n)$.

Tiempo: $O(n\,2^n)$
Espacio adicional: $O(n\,2^n)$

Igual que top-down. Pero bottom-up calcula todos los subconjuntos; top-down, sólo los alcanzables desde V.

Lo que veremos hoy

Introcucción y Metodología
- Problema de la Mochila

Motivación
- Fibonacci

Ejercicios teórica
- Varillas
- Producto de Matrices
- Conjunto Independiente Máximo

Ejercicios de la Guía
- OptiPago
- César

OptiPago: Repaso del Enunciado y Recurrencia

Tenemos un multiconjunto $B = \{b_1, \ldots, b_n\}$ y queremos pagar un costo C. Buscamos pagar con el mínimo exceso posible y, a igual exceso, usar la menor cantidad de billetes.

El estado lo definimos con i (considerando los primeros i billetes) y j (monto restante). La función recursiva devuelve tuplas (c, q):

$$OptiPago(i,j) = \begin{cases} (0,0) & \text{si } j \leq 0 \\ (\infty,\infty) & \text{si } j > 0 \text{ e } i = 0 \\ \min_{opt} \begin{cases} OptiPago(i-1,j) \\ OptiPago(i-1,j-b_i) \oplus b_i \end{cases} & \text{c.c.} \end{cases}$$

Llamada inicial: $OptiPago(n, C)$

* Donde $(c', q) \oplus b_i = (c' + b_i, q + 1)$ y $\min_{opt}$ minimiza el costo total primero y desempata por la cantidad de billetes.

¿Por qué la recursión minimiza el costo total? Como el costo C es un valor constante, minimizar el exceso (Exceso = Costo Total $- C$) es matemáticamente equivalente a minimizar el Costo Total.

OptiPago: Solución Top-Down

OptiPago-Mem(B, i, j)
  si $j \leq 0$: **devolver** $(0, 0)$
  si $i = 0$: **devolver** (∞, ∞)
  si $M[i, j] \neq \perp$: **devolver** $M[i, j]$
  $exc \leftarrow$ OptiPago-Mem$(B, i - 1, j)$
  $(c, q) \leftarrow$ OptiPago-Mem$(B, i - 1, j - B[i])$
  $inc \leftarrow (c + B[i], q + 1)$
  $M[i, j] \leftarrow \min_{opt}(exc, inc)$
  devolver $M[i, j]$

OptiPago: Backtracking vs. Top-Down

Recursión pura (Backtracking)

- Explora todas las combinaciones posibles sin recordar.
- **Tiempo:** $\mathcal{O}(2^n)$ (Exponencial). Depende de la cantidad de billetes n.
- **Espacio:** $\mathcal{O}(n)$.

Con Memoria (Top-Down)

- Memoizamos: cada estado (i, j) se calcula una única vez y se guarda en M.
- **Tiempo:** $\Theta(n \cdot C)$ (Pseudopolinomial). El cálculo por estado es $\mathcal{O}(1)$.
- **Espacio:** $\Theta(n \cdot C)$ (Para almacenar la matriz de estados).

Conclusión

¡La tabla salva el rendimiento ante el solapamiento masivo de subproblemas!

¡A pensar!

Consigna:

1. Intenten diseñar el algoritmo iterativo (Bottom-Up) para llenar la matriz de memoria M.
2. *Pistas:* Observen la fórmula recursiva. ¿Qué dimensiones debe tener la matriz? Si estamos en la fila i, ¿qué filas necesitamos tener calculadas previamente?
3. ¿En qué orden debemos iterar los ciclos `for`?

OptiPago: De Top-Down a Bottom-Up

- **¿Qué guarda la tabla?** Una matriz M de tamaño $(n+1) \times (C+1)$ donde $M[i, j]$ guarda la tupla óptima (c, q) utilizando los primeros i billetes para intentar cubrir un costo restante j.
- **Dependencias:** Para calcular los valores de la fila i, solo necesitamos mirar los valores ya computados de la fila $i-1$.
- **Casos base en la tabla:**
  - Para $i = 0$ y $j > 0$, inicializamos con $M[0, j] = (\infty, \infty)$.
  - Cualquier consulta donde $j \leq 0$ asume lógicamente el retorno $(0, 0)$.
- **Orden de llenado:** Iteramos i desde 1 hasta n (agregando billete a billete), y para cada i, iteramos el costo j desde 1 hasta C.

OptiPago: Solución Bottom-Up

```
OptiPago-BU(B, n, C)
  para j = 1, ..., C
      M[0, j] ← (∞, ∞)                          ▷ Imposible cubrir saldo sin billetes
  para i = 1, ..., n
      para j = 1, ..., C
          exc ← M[i − 1, j]                     ▷ Decisión 1: No usar el billete B[i]
          si j − B[i] ≤ 0 :                     ▷ Decisión 2: Usar el billete B[i]
              inc ← (B[i], 1)                   ▷ El billete cubre el saldo entero
          else
              (c, q) ← M[i − 1, j − B[i]]
              inc ← (c + B[i], q + 1)
          M[i, j] ← min_opt(exc, inc)
  devolver M[n, C]
```

OptiPago: Análisis de Complejidad

Tiempo (Bottom-Up)

- Llenamos una tabla de dimensiones $n \times C$.
- En cada celda realizamos operaciones $\mathcal{O}(1)$ (sumas básicas y una comparación $\min_{opt}$).
- **Complejidad Temporal:** $\Theta(n \cdot C)$

Espacio (Bottom-Up Optimizado)

- *Ingenuo:* $\Theta(n \cdot C)$ si guardamos toda la matriz.
- *Optimizado:* $\Theta(C)$
- Como para calcular la fila i dependemos **exclusivamente** de la fila $i-1$, **solo necesitamos guardar 2 vectores de tamaño** C (la fila anterior y la actual).

Lo que veremos hoy

Introcucción y Metodología
- Problema de la Mochila

Motivación
- Fibonacci

Ejercicios teórica
- Varillas
- Producto de Matrices
- Conjunto Independiente Máximo

Ejercicios de la Guía
- OptiPago
- César

Las legiones del César

Al famoso general César le gusta poner en fila sus tropas, que son patos y dodos (extraído de Codeforces 118D).

- Tiene un ejército de P patos y D dodos.
- No puede haber más de m_P patos consecutivos ni más de m_D dodos consecutivos.
- Los patos son indistinguibles entre ellos, al igual que los dodos.

Le interesa contar la cantidad de formas posibles de formar esta fila.

a) Definir una función recursiva $f_{m_P,m_D}(t, n_P, n_D, k_P, k_D)$ que cuente las formas de poner los patos y dodos restantes, si quedan t lugares, n_P patos y n_D dodos, y al final de la fila hay k_P patos y k_D dodos consecutivos. Pensar qué elección tomamos en cada paso.

b) Modificarla para que su firma sea $f_{m_P,m_D}(n_P, n_D, k, ultTropa)$, donde $ultTropa$ es la última tropa puesta y k la cantidad consecutiva de la misma.

c) Modificarla para que su firma sea $f_{m_P,m_D}(n_P, n_D, ultTropa)$.

d) Calcular la complejidad temporal y espacial de cada versión.

e) Demostrar la correctitud de cada versión.

f) **Optimizar la solución para que sea $O(P \cdot D)$ en tiempo y espacio (enfoque bottom-up).**